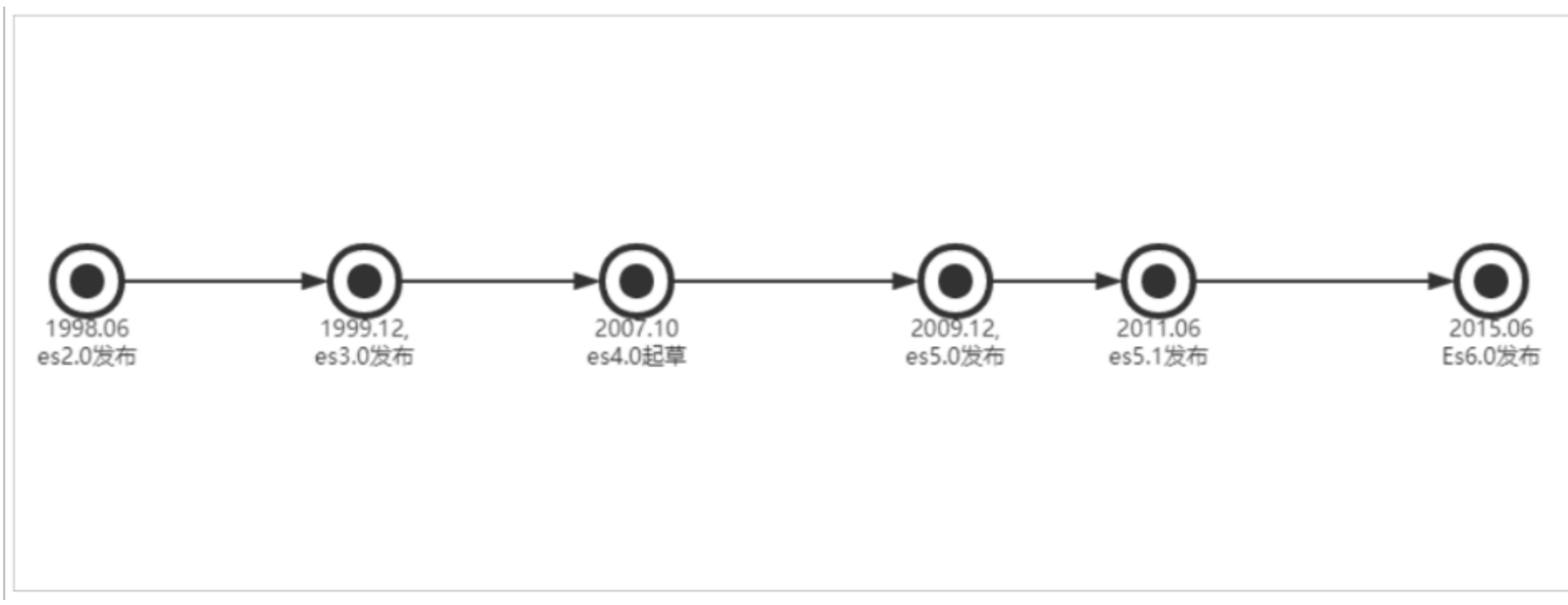
ES6详解

概述

ECMAScript的快速发展：



编程语言JavaScript是ECMAScript的实现和扩展 。ECMAScript是由ECMA（一个类似W3C的标准组织）参与进行标准化的语法规范。ECMAScript定义了：

- [语言语法] – 语法解析规则、关键字、语句、声明、运算符等。
- [类型]– 布尔型、数字、字符串、对象等。
- [原型和继承]
- 内建对象和函数的
- [标准库] – [JSON]、[Math]、[数组方法]、[对象自省方法]等。

ECMAScript标准不定义HTML或CSS的相关功能，也不定义类似DOM（文档对象模型）的[Web API]，这些都在独立的标准中进行定义。ECMAScript涵盖了各种环境中JS的使用场景，无论是浏览器环境还是类似[node.js]的非浏览器环境。

ECMAScript标准的历史版本分别是1、2、3、5。

那么为什么没有第4版？其实，在过去确实曾计划发布提出巨量新特性的第4版，但最终却因想法太过激进而惨遭废除（这一版标准中曾经有一个极其复杂的支持泛型和类型推断的内建静态类型系统）。

ES4饱受争议，当标准委员会最终停止开发ES4时，其成员同意发布一个相对谦和的ES5版本，随后继续制定一些更具实质性的新特性。这一明确的协商协议最终命名为“Harmony”，因此，ES5规范中包含这样两句话

ECMAScript是一门充满活力的语言，并在不断进化中。未来版本的规范中将持续进行重要的技术改进2009年发布的改进版本ES5，引入了[Object.create()]、[Object.defineProperty()]、[getters]和[setters]、[严格模式]以及[JSON]对象。

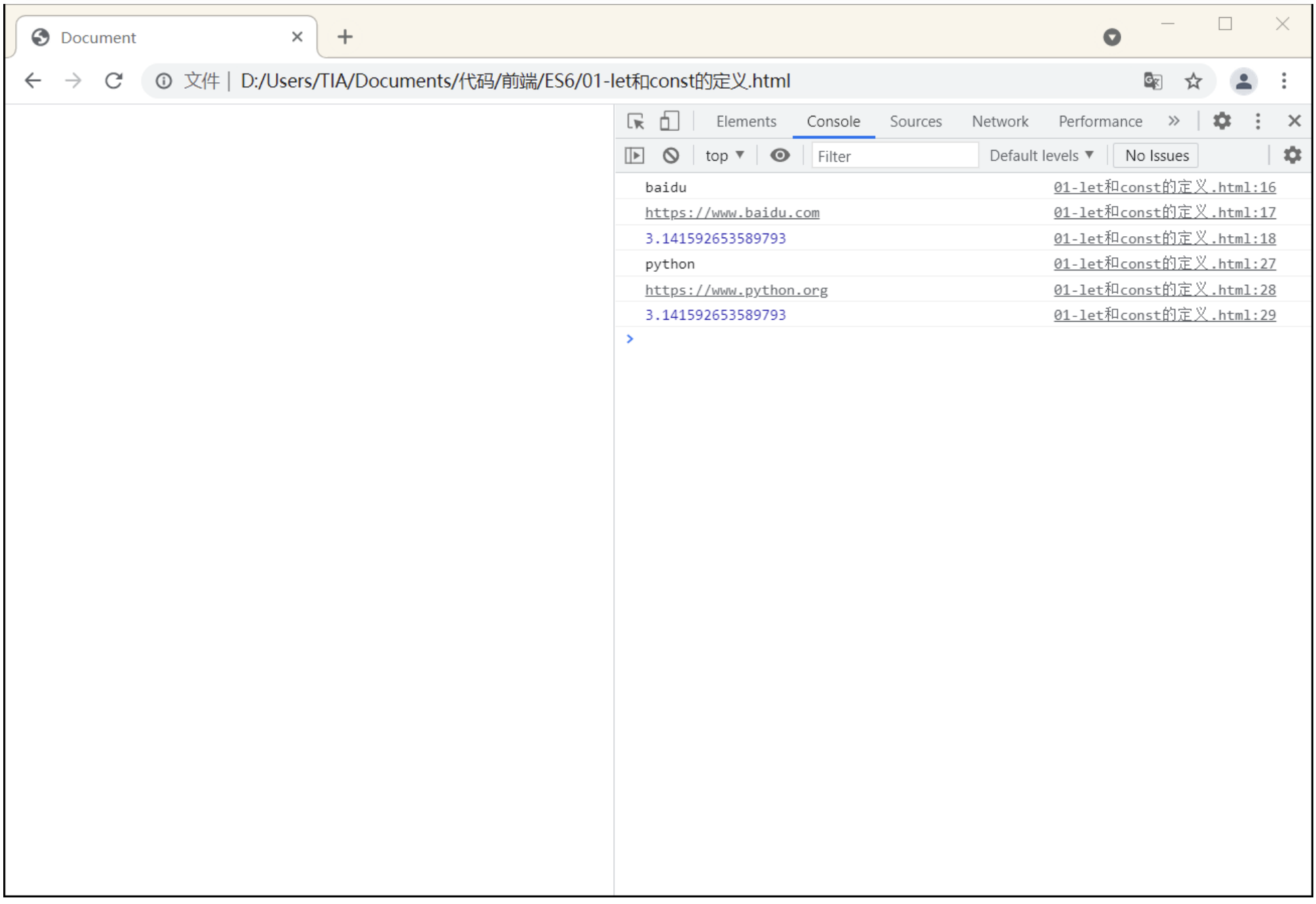
ES6: 是JavaScript语言的下一代标准，2015年6月正式发布。它的目标，是使得JavaScript语言可以用来编写复杂的大型应用程序，成为企业级开发语言。

语法

let和const命令

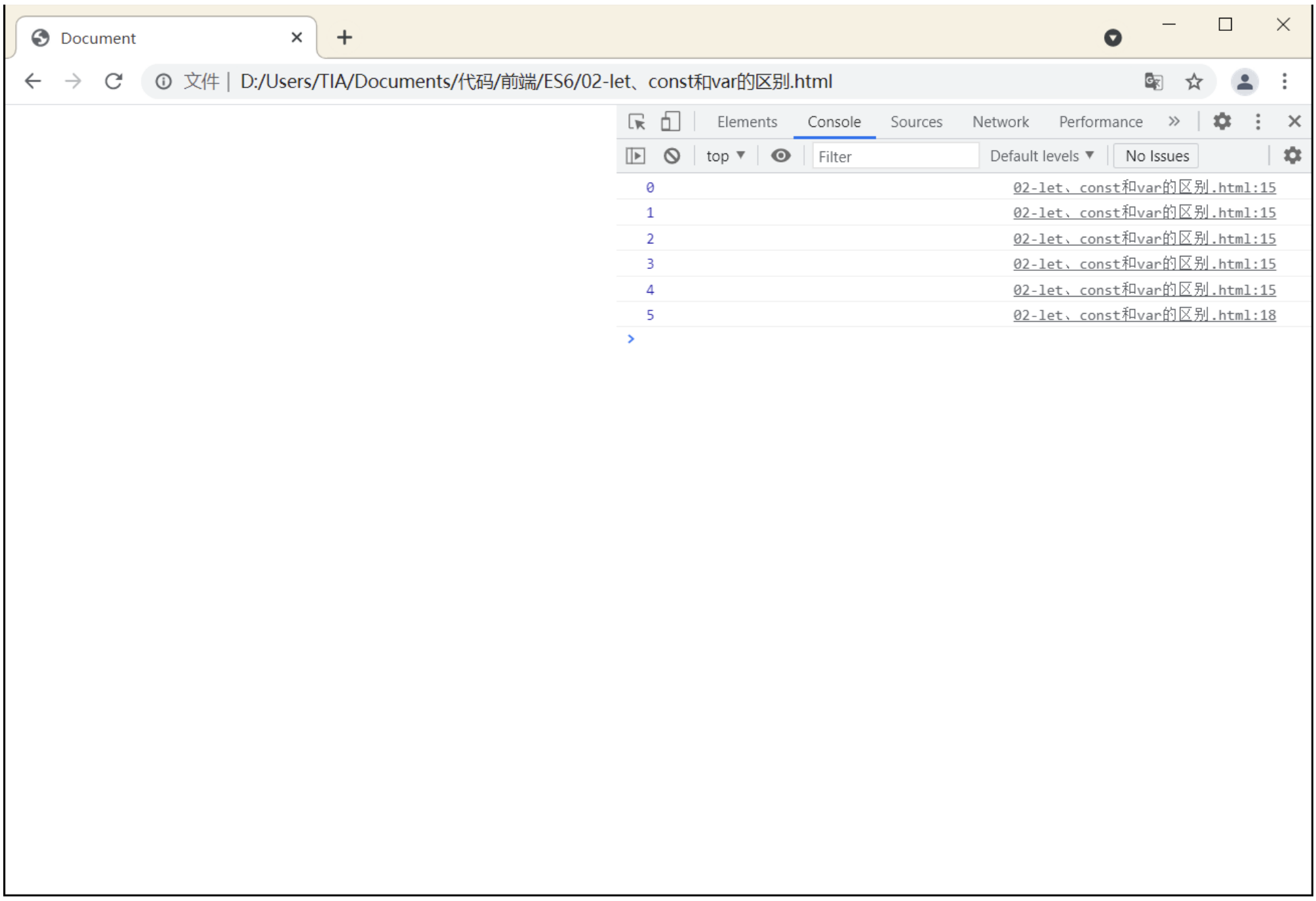
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <script>
        // 传统定义变量和常量的方式 统一使用var
```

```html
        var name = "baidu"
        var link = "https://www.baidu.com"
        var PI = Math.PI

        console.log(name)
        console.log(link)
        console.log(PI)

        // ES6 的定义
        let name2 = "python"
        let link2 = "https://www.python.org"

        // 定义常量
        const PI2 = Math.PI

        console.log(name2)
        console.log(link2)
        console.log(PI2)
    </script>
</body>
</html>
```



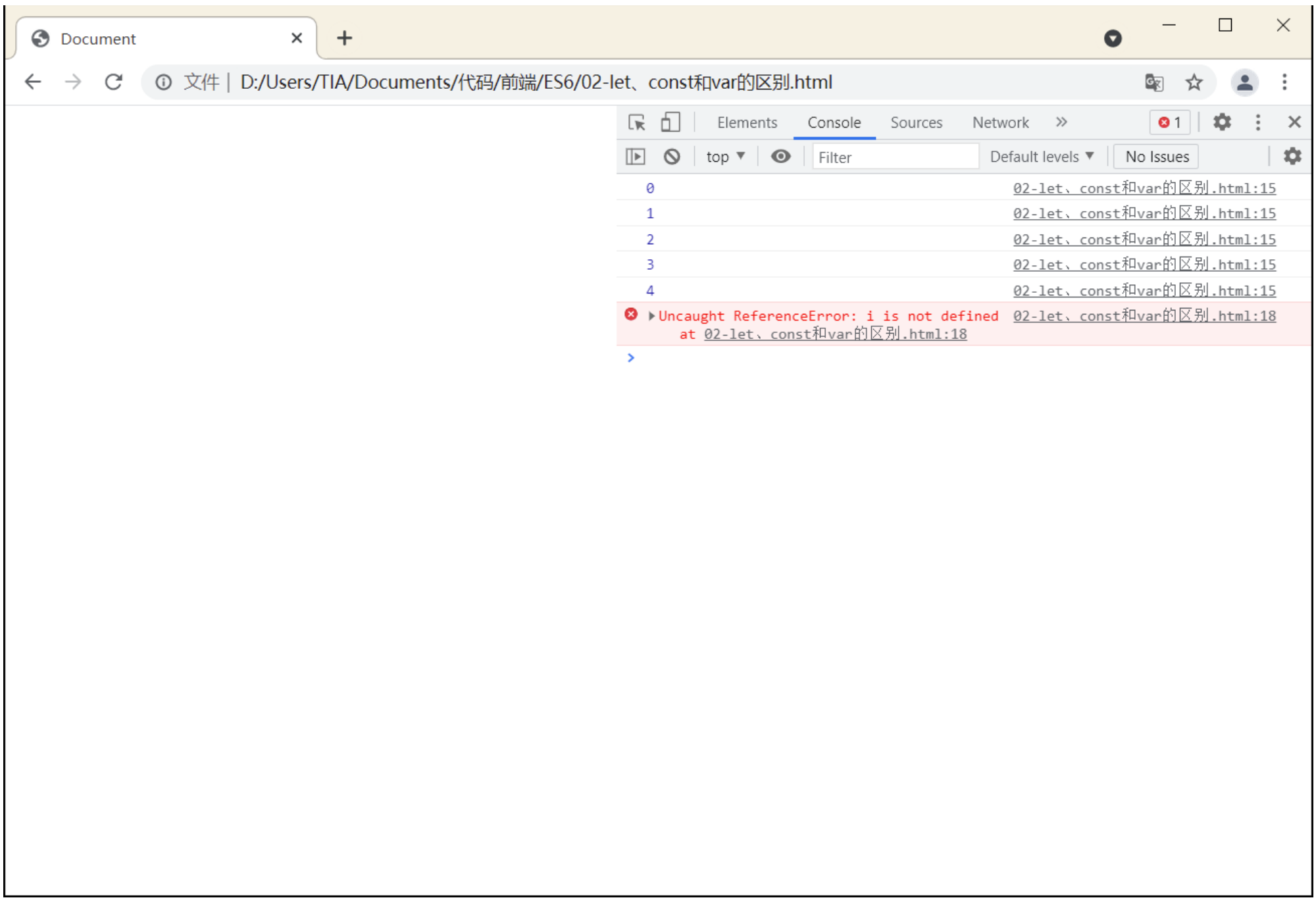
- var的变量穿透问题

```html
<!DOCTYPE html>
<html lang="en">
    ...
<body>
    <script>
        // var 的变量穿透问题
        for(var i=0;i<5;i++)
            console.log(i)

        // 这里就造成变量穿透、编译
        console.log(i)
    </script>
</body>
</html>
```



通过使用let进行解决

```html
<!DOCTYPE html>
<html lang="en">
    ...
<body>
    <script>
        for(let i=0;i<5;i++)
            console.log(i)
        console.log(i)
    </script>
</body>
</html>
```

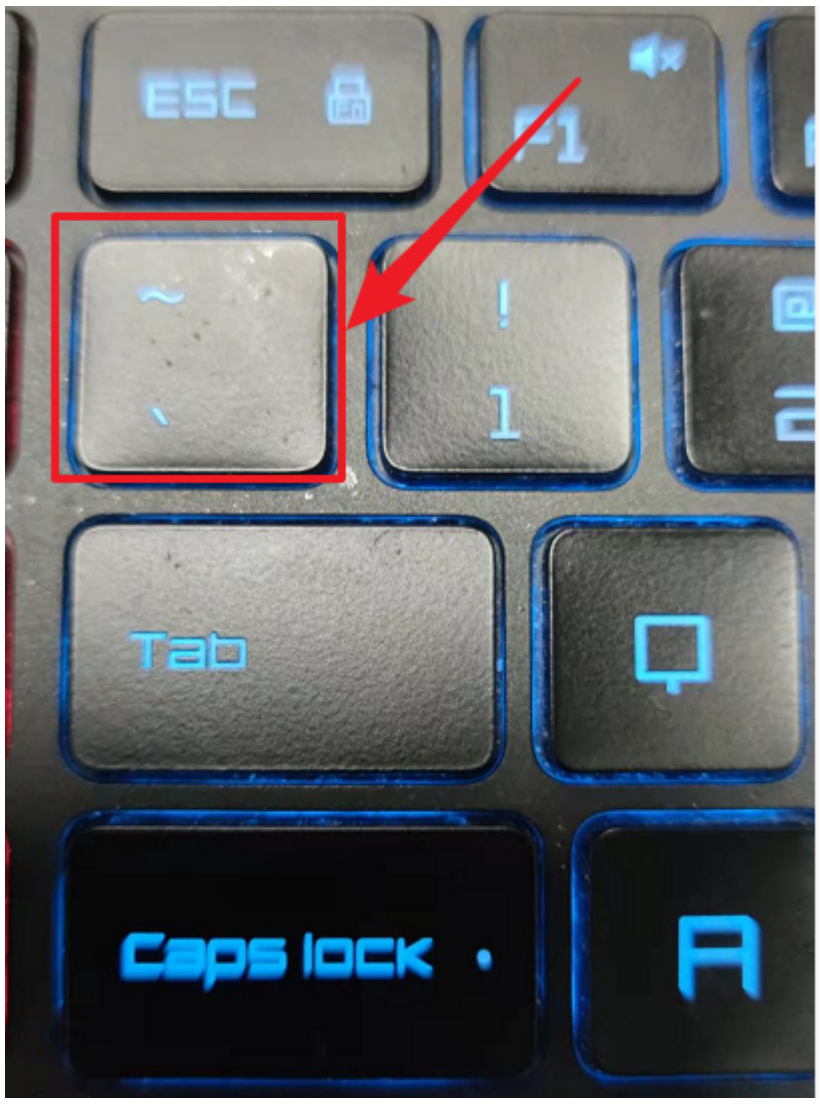


在实际开发和生产中，如果是小程序，uniapp或者一些脚手架中的，可以大胆的去使用let和const但是如果你是web开发。建议大家还是使用var，因为在一些低版本的浏览器还是不支持let和const。

模板字符串

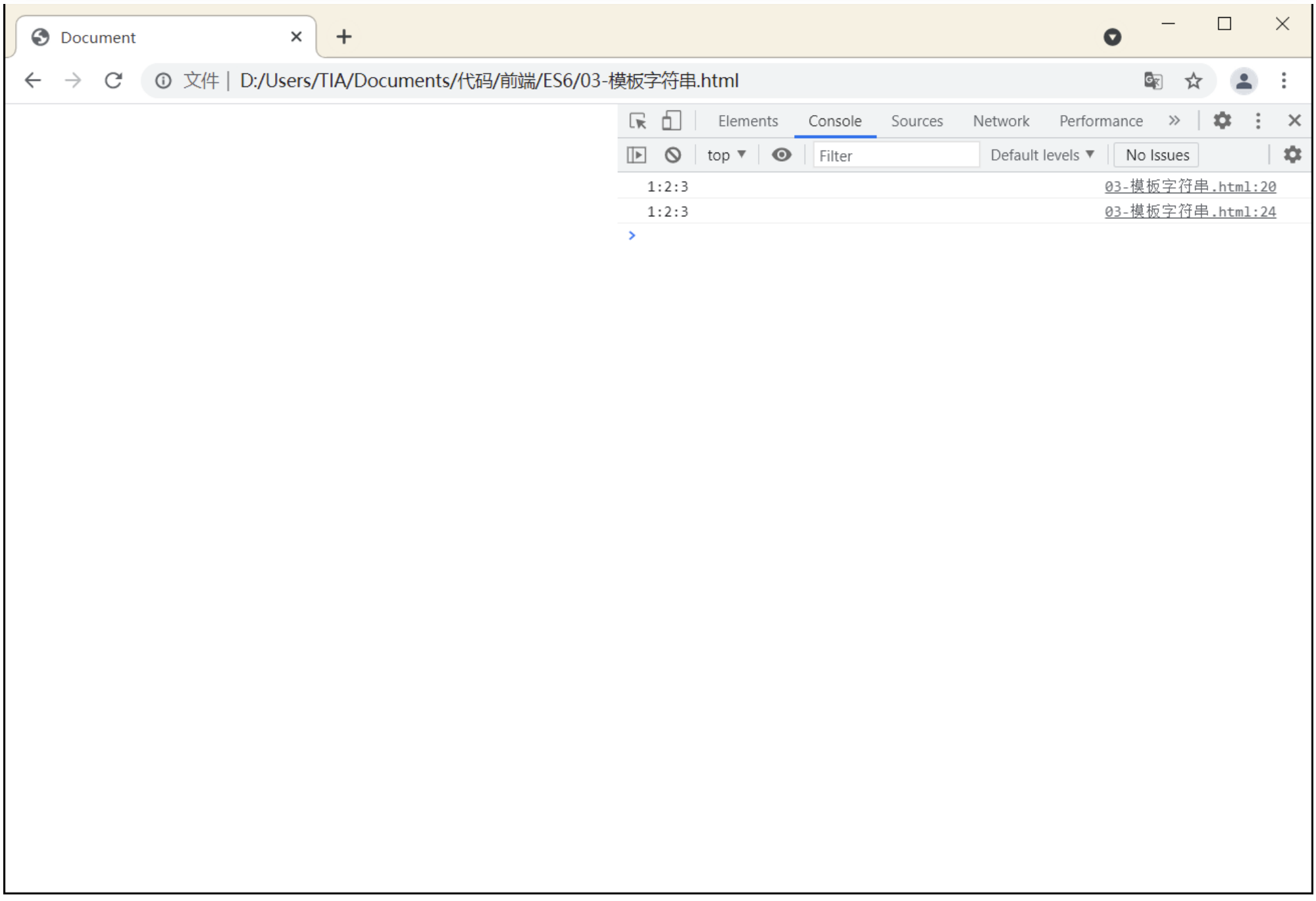
以前： 我们都是使用 ‘’ 或者 “” 来把字符串套起来

现在： ``【反引号】




```html
<!DOCTYPE html>
<html lang="en">
<head>
    ...
```

```
</head>
<body>
    <script>
        // 字符串会牵涉到动态部分

        var person = {
            name:"1",
            address:"2",
            link:"3"
        }

        let address = person.name + ":" + person.address + ":" + person.link
        console.log(address)

        // ES6字符串语法模板
        let address2 = `${person.name}:${person.address}:${person.link}`
        console.log(address2)

    </script>
</body>
</html>
```



函数默认参数与箭头函数

函数默认参数

```
<script>
    // 函数默认参数
    function sum(a,b){
        return a+b
    }

    var result = sum(1,2)
    console.log(result)
</script>
```

```
3                                                    ES6/04-函数默认参数.html:17
```

```
<script>
    // 函数默认参数
    function sum(a=2,b=3){
        return a+b
    }


    var result = sum()
    console.log(result)
</script>
```

```
5                                                    ES6/04-函数默认参数.html:18
```

箭头函数

箭头函数简化函数的定义，可以让我们不用使用function关键字。箭头函数最直观的三个特点：

1. **不需要function关键字来创建函数**
2. **省略return关键字**
3. 继承当前上下文的 **this** 关键字

```
<script>
    var sum = function(a,b){
        return a+b
    }

    var sum2 = (a,b) => {
        return a+b
    }

    var sum3 = (a,b) => a+b

     console.log(sum(1,2)+":"+sum2(1,2)+":"+sum3(1,2))
</script>
```

```
3:3:3                                                ES6/05-箭头函数.html:22
```

但是如果有逻辑体，就不能省略 `return` ，例如：

```
var sum = (a,b) => {
    var num = a + b
    return num
}

```

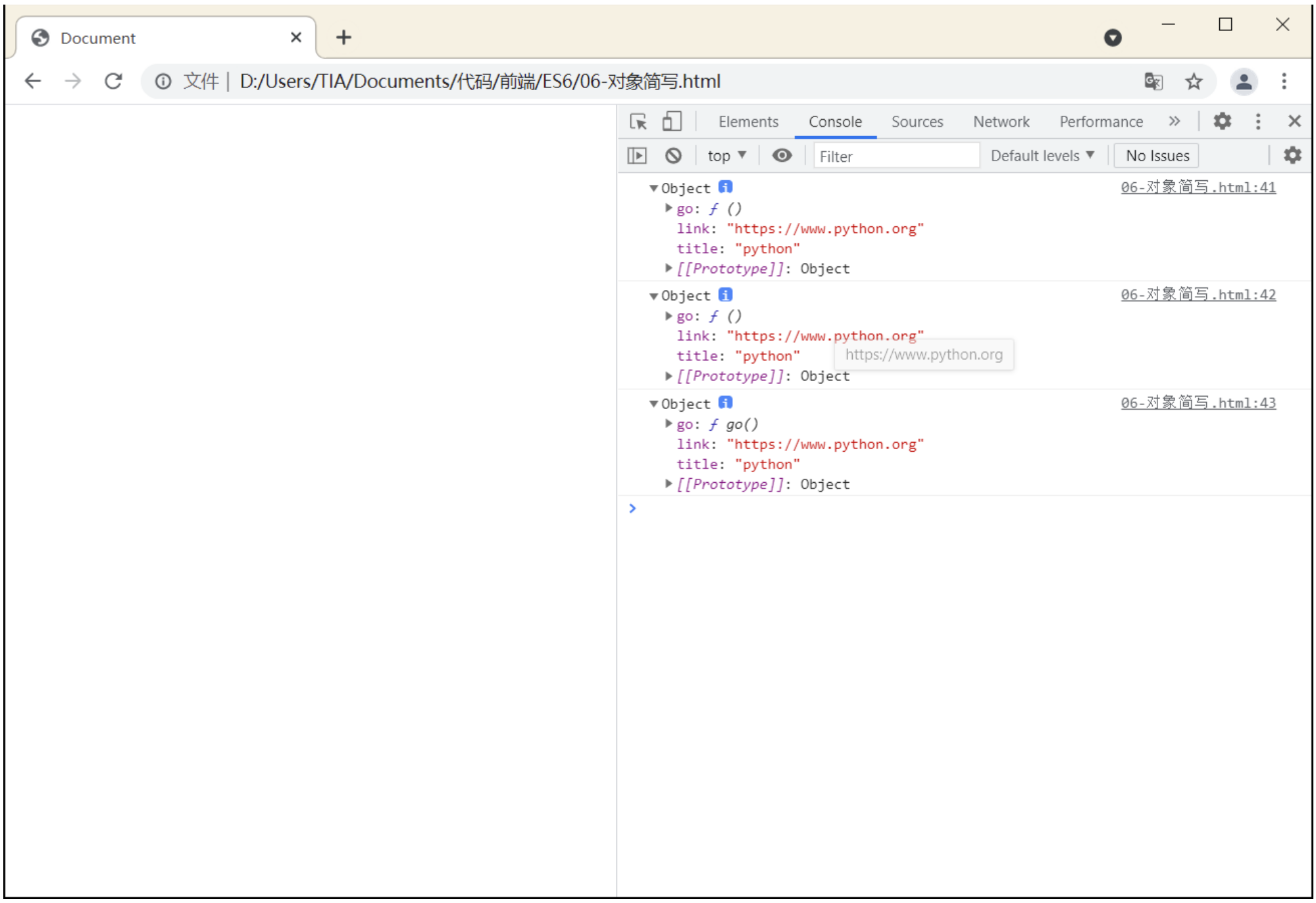
如果参数只有一个，括号也可以省略掉，例如：

```
<script>
    var arr = [1,2,3,4,5]
    var newarr = arr.map(obj=>obj * 2)
    console.log(newarr)
</script>
```

```
> (5) [2, 4, 6, 8, 10]
```

对象初始化简写

```
<script>
    var info = {
        title:"python",
        link:"https://www.python.org",
        go:function(){
            console.log(`正在学习${this.title}`)
        }
    }

    //es6简写
    //因为对象是以key-value的形式存在
    //1. 如果key和变量名字一致，可以指定义一次即可
    //2. 如果value是一个函数，可以把`:function`去掉，只剩下()即可
    var title = "python"
    var link = "https://www.python.org"
    var info2 = {
        title:title,
        link:link,
        go:function(){
            console.log(`正在学习${this.title}`)
        }
    }

    var info3 = {
        title,
        link,
        go(){
            console.log(`正在学习${this.title}`)
        }
    }

    console.log(info)
    console.log(info2)
    console.log(info3)



</script>
```



对象结构

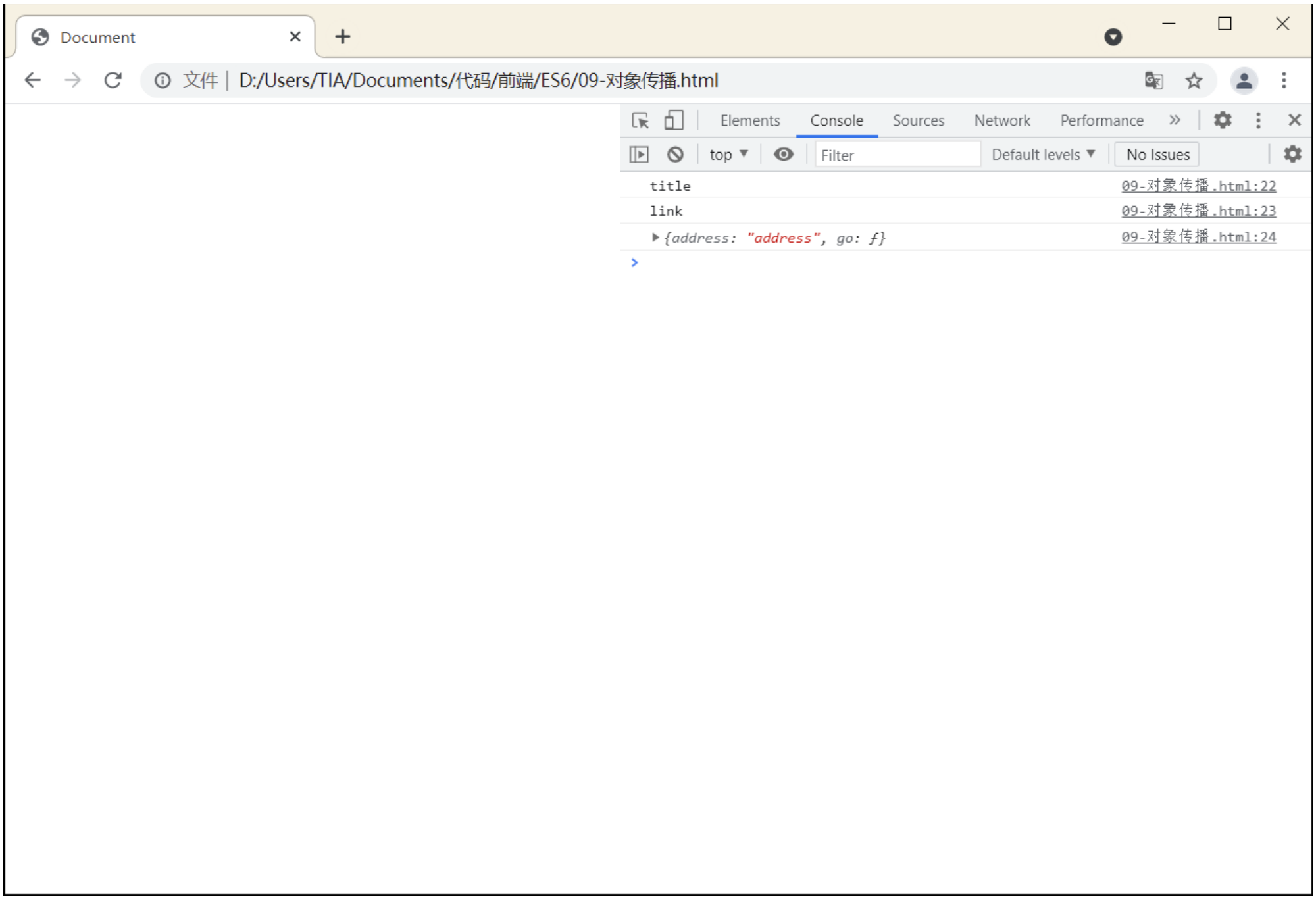
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>对象解构</title>
</head>
<body>
    <script>
        // 对象解构 --- es6提供一些获取快捷获取对象属性和行为方式
        var person = {
            name:'zhangsan',
            age:32,
            language:"cn",
            // 函数也有处理
            /* say:function(){
                console.log(this.name+"年龄是: " + this.age);
            }   */
            /* say:()=>{
                console.log(this.name+"年龄是: " + this.age);
            }  */
            say(){
                console.log(this.name+"年龄是: " + this.age);
            }
        };
        // ==========================传统的做法========================
        var name = person.name;
        var age = person.age;
        person.say();
        // ==========================对象解构做法========================
        //es6的做法 前提: 默认情况name,age必须是jsonkey.
        var {name,age} = person;
        console.log(name,age);
        // 可以用冒号取小名
        var {name,age,language:lan} = person;
        console.log(name,age,lan);
    </script>
```

```
</body>
</html>
```

传播操作符

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <script>
        // 对象传播操作符
        var info = {
            title:"title",
            link:"link",
            address:"address",
            go:function(){
                console.log(`${this.title},${this.link}`)
            }
        }

        var {title, link,...person2} = info
        console.log(title)
        console.log(link)
        console.log(person2)


    </script>
</body>
</html>
```

从对象中进行取值，取值的名字需要与对象中的名字一致；当对对象进行取值时，如果剩下还有值，可以通过 `...xxx` 进行获取，如 `...person2` 。



数组map和reduce方法使用

map

方法可以将原数组中的所有元素通过一个函数进行处理并放入到一个新数组中并返回该新数组。

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <script>
        // 要对数组每个元素*2
        let arr = [1,2,3,4,5,6,7]

        // 传统的方式
        let newarr = arr.map(function(value){
            return parseInt(value) * 2;
        })

        // 传统的方式
        let newarr2 = arr.map(value=>{
            return parseInt(value) * 2;
        })

        // 传统的方式
        let newarr3 = arr.map(value=>parseInt(value) * 2)

        console.log(newarr)
        console.log(newarr2)
        console.log(newarr3)

    </script>
</body>
</html>
```

reduce

reduce(function(),初始值（可选））：

接收一个函数（必须）和一个初始值（可选），该函数接收两个参数：

- 第一个参数是上一次reduce处理的结果
- 第二个参数是数组中要处理的下一个元素
  reduce() 会从左到右依次把数组中的元素用reduce处理，并把处理的结果作为下次reduce的第一个参数。如果是 第一次，会把前两个元素作为计算参数，或者把用户指定的初始值作为起始参数

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <script>
        let arr = [1,2,3,4,5]
        var result = arr.reduce((a,b) => a+b) # result=15
        console.log(result)
    </script>
</body>
</html>
```